

SUMMARY

RAG evidence selection: coverage 66.25% → 91.25%, false evidence 20 → 0

Agent execution environment: user-defined skills behind approval, edit quality 82.5% → 91.2% (114 drafts)

Data pipelines: PDF conversion pass rate 45.17% → 89.89%, drug conversion \$200 estimated → \$11.18 spent

PROJECTS & EXPERIENCE

Fruition

2026.04 — Present

LLM Wiki-based AI workspace · AI SW Maestro, 17th cohort · Deployed on AWS EKS, in QA

Team project (3 + designer) / AI lead

How it started: a teammate's pain point with managing documents, refined through design thinking — saved documents could not be found when needed.

Team: me (all AI), 1 backend (Spring backend), 1 frontend & DevOps (frontend, MSA deployment on AWS), external designer · 3 mentors

[GitHub · AI repository ↗](#)

Improving retrieval for grounded AI answers

Finds the document evidence AI answers need, and attaches none to questions the documents cannot answer.

- Evidence selection for RAG

Problem: In a service that answers from documents, retrieval missed evidence and attached evidence to unanswerable questions, undermining trust.

Decision: GPT selection cost about 2.5x and took longer; tuning weights and RRF gave similar scores across settings, so I adopted Jev, which picks only from fixed options.

Solution: Two stages: BGE-M3 and BM25 gather candidates and Jev picks the final evidence. Compared on identical candidates with blind cross-grading, then tuned to 300 candidates and 4 parallel batches.

Result: Coverage on 80 answerable questions rose from 66.25% to 91.25%. Median selection time 9.090s → 2.251s (still slower than the original 0.431s).

Agent execution environment - skills, approval, routing and conversation context

Designed reusable Skills with user approval before changes. Applied Jev to request routing, and set the scope of the conversation context and edit input the model receives.

- User-defined task instructions (Skills)

Problem: Recurring tasks differ by user, so they cannot all be built as features, and AI edits arrived with parts of the request missing, leaving repeatable work to chance.

Decision: AI drafts instructions, code checks permissions and scope, and the user decides publishing and applying.

Solution: Authoring, review and publishing of natural-language instructions with checks against bypassing permission or approval; preview and separate approval before changes; semantic evaluation with at most one rewrite.

Result: Edit quality 94 → 104 of 114 drafts (82.5% → 91.2%); documents stayed unchanged before approval.

- Jev-based request routing

Problem: Misrouting a question as an edit led to the wrong action, directly hurting the service's reliability.

Decision: Jev picks only from fixed options, which fits routing; as a newly released model it was validated first on low-risk routing.

Solution: Applied Jev to choose task type and conditions, and compared it with the existing approach and GPT-5-nano on the same 98 requests.

Result: Fully correct decisions rose from 77 to 81 (GPT-5-nano 63); median decision time fell from 8.967s to 0.658s.

- Conversation memory and edit input scope

Problem: Sending whole conversations and documents to the model raises cost and latency.

Solution: Keep the last six messages verbatim with a rolling summary for older ones, and send only the target lines plus context for edits, cutting input for a 1,000-line document to 5.58% of the source.

Execution reliability and LLM tracing

Undoing changes an AI task already saved, and making each stage of a run inspectable. Execution spans several stores and cannot be one transaction, so recovery runs as reverse actions.

- Undoing a cancelled task in reverse

Problem: an AI task saves documents, folders, wiki pages and skills between model calls, so stopping execution leaves some changes behind.

Solution: persist the cancellation first to block later writes, then undo children first using a pre-change snapshot and an idempotency key tied to the approval. Recovery uses the run record alone, never a model call, and cancellation is final only once full recovery is confirmed.

- Recovering without erasing other tasks' contributions
Problem: a concept page merges contributions from several documents, so an older snapshot would erase valid contributions from other documents.
Solution: record operations, revisions and per-document contributions append-only, then reassemble with only the target contribution excluded. Soft delete instead of removal keeps the audit trail.
- LLM run tracing and the boundary around sensitive data
Problem: the evaluation loop ran as plain Python, so stages could not be inspected separately, and whatever reached the tracing tool could carry sensitive data.
Solution: rebuilt the loop as a LangGraph graph so generation, evaluation and retry each trace as a node. The agent graph keeps tracing off and excludes raw tool lookups from persisted state, and my own prompt log masks numeric personal data before writing. Timing runs disable tracing and logging entirely.

Document conversion pipeline and faster Wiki generation

Converted uploaded PDFs for retrieval and grounded answers, and reduced Wiki generation time.

- PDF → Markdown conversion
Problem: Broken tables and equations in paper PDFs made the figures behind Wiki pages and answers unusable.
Decision: Repairing only detected damage missed errors, and full AI conversion lacked table/equation accuracy and was costly, so I split processing by content type. Body text uses fast AnyDoc extraction; repair moved from an sLLM to commercial LLM APIs to avoid GPU server costs.
Solution: Extracted body text with AnyDoc and repaired only garbled parts with AI, read table/equation regions separately, and kept original figures.
Result: Pass rate on the same 30 pages (445 units, blind) rose from 45.17% to 89.89%.
- Parallel Wiki generation
Problem: Documents were processed one at a time, so large uploads waited long for their Wiki.
Decision: Compared worker count, concurrent requests and GPU parallelism. More workers helped while waiting on external AI; more concurrent requests and GPU parallelism were slower, so I dropped them. Accepted about 4x memory, since EKS can spread services.
Solution: Distributed per-document jobs over Kafka with up to 4 workers.
Result: Analysis of 4 documents fell from 282.11s to 73.89s (about 74% faster).

MSA: Splitting only by feature would keep DB load the same while adding hops, so I proposed splitting by data ownership and co-designed boundaries and communication with teammates and mentors.

Internal evaluations. RAG timing compares Jev before/after tuning. Parallel document timing covers analysis only, excluding storage and post-processing.

See the detailed portfolio for the problem-solving process and experimental results.

[RAG evidence selection](#) ↗ [Agent execution environment](#) ↗ [PDF conversion](#) ↗ [Parallel document processing](#) ↗

Pilltip

2025.03 — 2025.12

Personalized AI medication service · Pusan National University capstone · 6 users on internal APK builds
 Team project (3 + designer) / Core backend · data preparation

How it started: drug information is scattered, making it hard to check whether a medicine is safe for your current health condition.

Team: me (backend & AI), 1 Android (Kotlin app), 1 backend & security (user DB, login, questionnaires, AES-GCM), designer. The features below are my work.

[GitHub repository](#) ↗

Drug data preparation pipeline

Rewrote about 44,000 drug records in plain language, converting each repeated sentence once and reusing it per drug.

- Problem:** Rewriting about 44,000 drug records as-is was estimated at about \$200 in API costs, more than the budget allowed for covering every drug.
Decision: Instead of covering fewer drugs, convert each repeated sentence once. The job needed no real-time response, so Batch API traded waiting time for lower cost.
Solution: Split text into sentence blocks, merged exact duplicates, converted via GPT Batch API and mapped results back to each drug. Reproduced in PySpark in 2026 to measure each step.
Result: Actual API spending was \$11.18 versus about \$200 estimated for all original text. In the reproduction, exact deduplication cut text from 744MB to 44.1MB (-94.1%).

Estimated cost versus actual spending; preprocessing labor is excluded.

AI medication chatbot using health profiles

Found drug and medication information from natural-language symptoms. Evaluated health profiles internally, separately from information sent to external AI.

- Problem:** Users checking whether a drug is safe describe symptoms in different words from drug indications, and the health data needed for personalization could not go to external AI.
Decision: A synonym dictionary had too many overlapping symptoms, so I used semantic search; filtering the profile had too many cases, so profile decisions stay internal.
Solution: Weaviate semantic search and AI orchestration route to drug search, medication risk and dosage paths; pregnancy, current medications and underlying conditions are judged in internal code.
Result: Personalized symptom-based search with those profile fields excluded from external LLM inputs.

Backend feature implementation

Built server features for drug search, risk information, medication reminders and logs, and family profiles. Integrated with teammates responsible for the mobile app,

users and security.

- Designed the drug database on Java and Spring Boot with search, autocomplete and DUR risk display. Elasticsearch handles partial matching and autocomplete and Redis caches repeated risk lookups, cutting search responses from 5–7s to 0.8s on average.
- Built FCM medication reminders and logs, deep-link friend invites and family profile switching, and kept API docs, meeting notes and ADRs in Notion to share decisions.

Feature work driven by feedback from live exhibitions

We ran the booth twice as Pusan National University's representative exhibition team, demonstrating to more than 100 visitors and collecting 10 pieces of feedback. Most asked for health supplements, so I built that within the data we already had.

- Extending support to health functional foods
Problem: at the first exhibition visitors could only check prescription drugs and asked to see health functional foods too.
Solution: within the data we held, added them to search and autocomplete, surfaced caution and co-administration information, allowed them in reminders and logs, and included them in chatbot answers.
Result: a request from the floor became the next slice of work. Limited to the data we had obtained, not the exhaustive coverage we have for drugs.

See the detailed portfolio for the problem-solving process and experimental results.

[Data pipeline case study ↗](#) [Personalization case study ↗](#)

Skills

AI & Search	LangChain · LangGraph · Spring AI · RAG · LLM Evaluation · Prompt injection defense LangSmith · Elasticsearch · Weaviate
Backend	Java · Spring Boot · Python
Data & Infra	MySQL · PostgreSQL · Redis · Kafka Docker · Docker Compose · GitHub Actions

Activities & contributions

2026.07 — 2026.09	200 SIGNAL · TQQQ 200-day moving average signal app (personal project) I turned a publicly posted TQQQ 200-day moving average strategy into a written spec and an Android app on live quotes, used by 2 family members. I marked the source post's claimed returns as unverified and reproduced 45 cycles, measuring a 40.00% win rate rather than the claimed 19%. Counting partial exits as trades lifts that to 72.73%, showing the post's two win rates count different units. I backtested 5 strategy variants under the same assumptions, comparing CAGR and MDD across sub-periods. At the users' request I added a second QQQ-based strategy and SGOV/SPYM handling for cash waiting on an entry. The app was never released, and account data stays on the device. GitHub repository ↗
2026.06 — 2026.06	Rhwp open-source contributions Gave Claude Code a work plan and contribution rules to automate the flow from issue analysis to PR, including HWP/HWPX save-error fixes; 7 PRs merged and listed as a contributor. GitHub repository ↗ PR #1213 · Merged ↗ PR #1351 · Submitted ↗
2025.03 — 2026.01	 APPTIVE, Pusan National University Backend mentee and mentor. Led 6 mentoring and code-review sessions for 12 mentees. Helped revise the curriculum to strengthen HTTP, Servlet, REST API and database foundations. APPTIVE Backend Mentor Merit Award · 2026.01.09
2025.11 / 2025.07	 SK AI SUMMIT · K-ICT WEEK in Busan Represented Pusan National University at the Pilltip booth, demonstrating the service to more than 100 visitors and answering technical questions. Most of the 10 pieces of feedback asked for health supplement coverage.
2022.02 — 2025.07	945-day Baekjoon problem-solving streak Longest streak of 945 consecutive days on solved.ac. 1,659 problems solved, Platinum IV. Baekjoon solutions repository ↗

Education & certification

2022.02 — 2026.02 (Transfer)	Pusan National University GPA 4.12 / 4.5
2019.03 — 2022.02 (Withdrew)	Daegu University GPA 4.2 / 4.5

Awards

2025.10		Capstone Design Gold Award Software & AI division / College of Information and Biomedical Engineering, Pusan National University
2025.09		Busan DATA WEEK Top Excellence Award Data utilization case competition / Busan Techno Park
2025.09		AI LAUNCH Career School Startup Hackathon Encouragement Award KRYPTON X x Root Impact x Google.org
2025.08		SW University Digital Competition Sponsor Award SW University Council